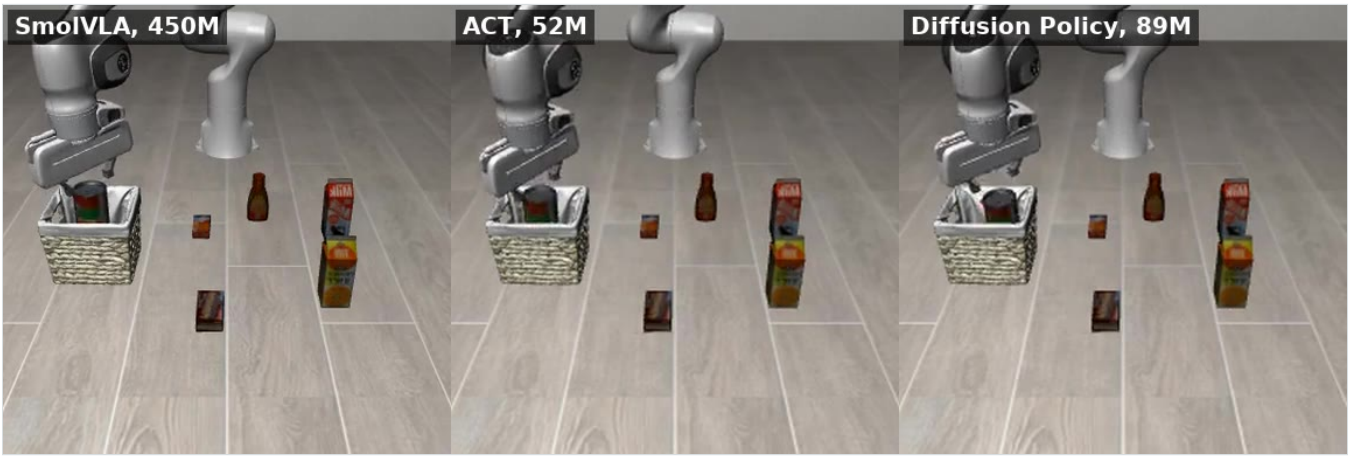
Study report · Nishanth Sundaran · 2 November 2026 · github.com/NishanthSundaran/libero-imitation-study

LIBERO Imitation Study Guide

How robots learn manipulation from demonstrations, explained through one complete project: three policies trained or reproduced on the same benchmark, compared fairly, with every failure looked at on video.

This guide is written to be studied. Each idea is explained first, then shown in the measurements from the project. Read it in order the first time; afterwards the interview section and glossary work on their own.



The same held-out episode (tomato sauce, init state 10, seed 3000) run by the three policies. All three succeed here; across 300 such episodes they do not succeed equally often.

98.3 % vs 94.3 % ACT (52M, trained here) vs the published SmolVLA (450M), 300 held-out episodes	**78 → 97 / 100** ACT success before and after image augmentation, same held-out episodes
70 · 90 · 97 ACT held-out success with 10, 20 and all ~45 demonstrations per task	**85 · 96 · 98** Diffusion Policy on the same 100 scenes with three different seeds

Contents

1. The problem: learning from demonstrations 3
2. Core ideas you need 4
3. The three policies 6
4. Evaluating fairly 8
5. What happened, step by step 10
6. Final results 12
7. How much data is enough 14
8. How the policies fail 15
9. Lessons that transfer 16
10. Interview questions 17
11. Glossary 19
12. Further reading 20

1. The problem: learning from demonstrations

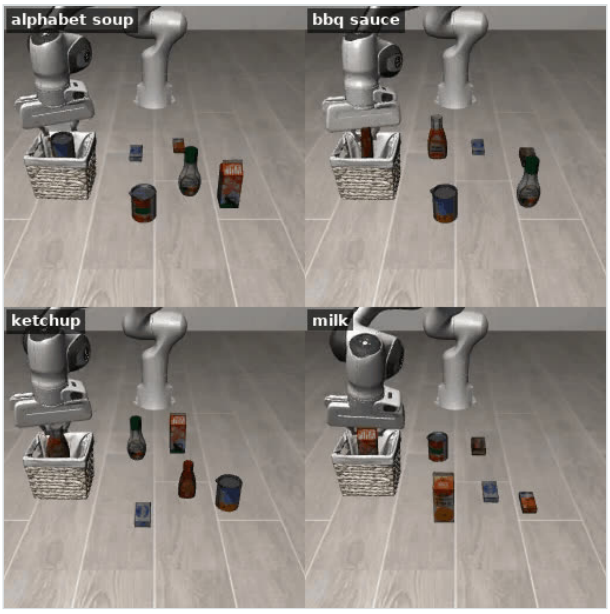
Imitation learning teaches a robot a skill by showing it examples instead of writing the motion by hand. A person teleoperates the robot through the task many times; each run is recorded as a sequence of camera images, joint readings and the commands that were sent. A neural network, the **policy**, is then trained to predict the command a demonstrator would send given what the robot currently sees. The simplest form, used here, is **behaviour cloning**: plain supervised learning from observation to action.

It sounds easy and is not, for three reasons that come back throughout this guide.

- **Small errors compound.** The policy only saw states a good demonstrator visited. A slightly wrong action moves the robot into a state it never saw, where its next action is worse, and so on. This is called covariate shift or compounding error.
- **Demonstrations are not consistent.** People grasp the same bottle from the left one time and from the right another time. A network trained to output the average of two good grasps can output a bad grasp in the middle. This is the multimodality problem.
- **Precision is unforgiving.** A grasp that is two centimetres off closes on air. Most failures in this project are exactly that.

The benchmark: LIBERO-Object

LIBERO is a standard simulated benchmark for robot manipulation, built on MuJoCo with a Franka Panda arm. It has four suites; this project uses **LIBERO-Object**: ten tasks of the form "pick up the *X* and place it in the basket", where *X* is alphabet soup, cream cheese, salad dressing, bbq sauce, ketchup, tomato sauce, butter, milk, chocolate pudding or orange juice. Each task has 50 official starting layouts called **init states**, which place the objects at different positions.



Four of the ten tasks, each finished by the same Diffusion Policy model. Every task has its own set of objects on the table, so the scene alone tells a policy which object to pick.

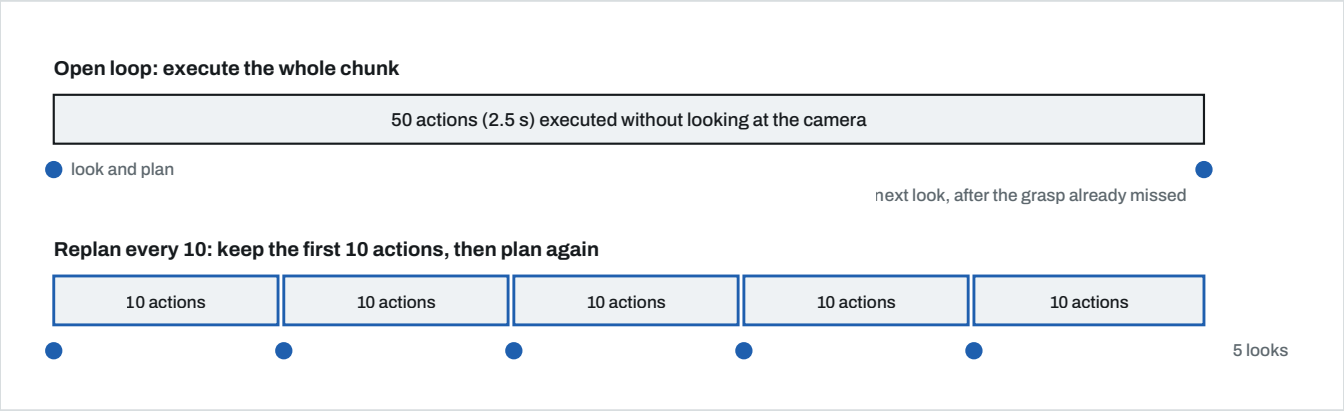
What the policy gets and gives	Details
Images	Two cameras, 256 × 256 px: a fixed view of the table and a camera on the wrist
Robot state	8 numbers: gripper position and orientation, and the two finger positions
Action	7 numbers per step: a small move and rotation of the gripper, plus open or close
Rate and time limit	20 actions per second; an episode fails if the object is not in the basket after 280 steps (14 s)
Training data	454 demonstrations for the ten tasks (42 to 50 per task), from the LeRobot LIBERO dataset

2. Core ideas you need

Action chunking

Instead of predicting one action per camera frame, modern policies predict a **chunk**: the next 16, 50 or 100 actions at once. Chunking makes the effective task shorter (fewer decisions means fewer

chances for errors to compound) and copes better with the pauses and hesitations in human demonstrations. The cost is that a chunk is a plan made with old information.



Same 2.5 seconds, two ways to execute a predicted chunk. Replanning still predicts a full chunk each time but throws most of it away, so a missed grasp is noticed within half a second.

There are three ways to use a chunk, and this project measured all three.

- **Open loop:** execute every predicted action, then look again. Simple and smooth, but blind for the whole chunk.
- **Replanning (receding horizon):** execute only the first few actions, then predict a fresh chunk. Replanning every 10 actions was the single most valuable change at evaluation time.
- **Temporal ensembling:** predict every step and average all the overlapping chunks that cover the current step. ACT's paper recommends it, but here it hurt badly (45/100 against 76/100), because averaging smooths away the small last-moment corrections a precise grasp needs.

Match the training conditions at test time

A policy is only reliable on inputs that look like its training data. Two mismatches in this project silently cost a lot of success: images rendered at a different resolution than the training frames (ACT does not resize its input, so 360 px images produce a different feature map than the 256 px frames it learned on), and a different simulator version that renders the floor darker. Neither raises an error; the success rate just drops.

Overfitting looks like progress

Training loss measures how well the policy reproduces the demonstrations it has already seen. Success measures whether it completes the task in new layouts. When training longer keeps lowering the loss while success stays flat or falls, the policy is memorising exact pixel positions instead of learning the skill. **Image augmentation** (small random shifts, rotations and colour changes applied to training images) makes memorising pixels impossible and forces the network to learn what matters.

3. The three policies

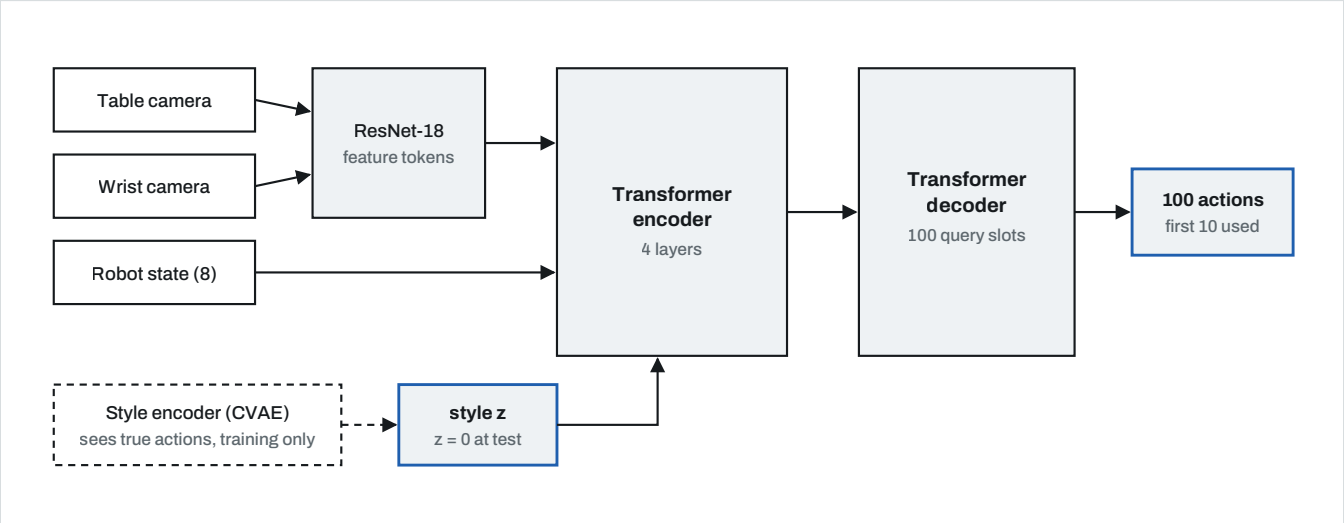
All three take camera images and robot state and output a chunk of actions. They differ in how they produce that chunk, and that difference explains most of what happened in the experiments.

	ACT	Diffusion Policy	SmolVLA
Parameters	52M	89M	450M
How it makes a chunk	One forward pass of a transformer	Starts from random noise and removes it in 10 steps	Vision-language backbone plus an action expert that integrates a flow in 10 steps
Chunk predicted / used	100 / 10	16 / 8	50 / 10
Uses language	No	No	Yes, the task instruction
Random at test time	No	Yes	Yes
Who trained it	This project	This project	Published checkpoint from its authors

ACT: Action Chunking with Transformers

ACT (Zhao et al., 2023) was designed for precise two-arm manipulation from a small number of demonstrations. Each camera image goes through a ResNet-18 that turns it into a grid of feature tokens. A transformer encoder mixes those tokens with the robot state and one extra token, the style variable z. A transformer decoder then turns 100 learned query slots into 100 future actions in a single pass. Training minimises the L1 distance between predicted and demonstrated actions.

The style variable is ACT's answer to inconsistent demonstrations. During training a second, small encoder looks at the true future actions and summarises "how this demonstrator did it" into z, so the main network does not have to average different styles. This is a **conditional variational autoencoder** (CVAE); a KL penalty keeps z close to a standard normal distribution. At test time there is no demonstrator, so z is set to zero, the average style. That is why ACT is **deterministic**: the same observation always gives the same chunk.

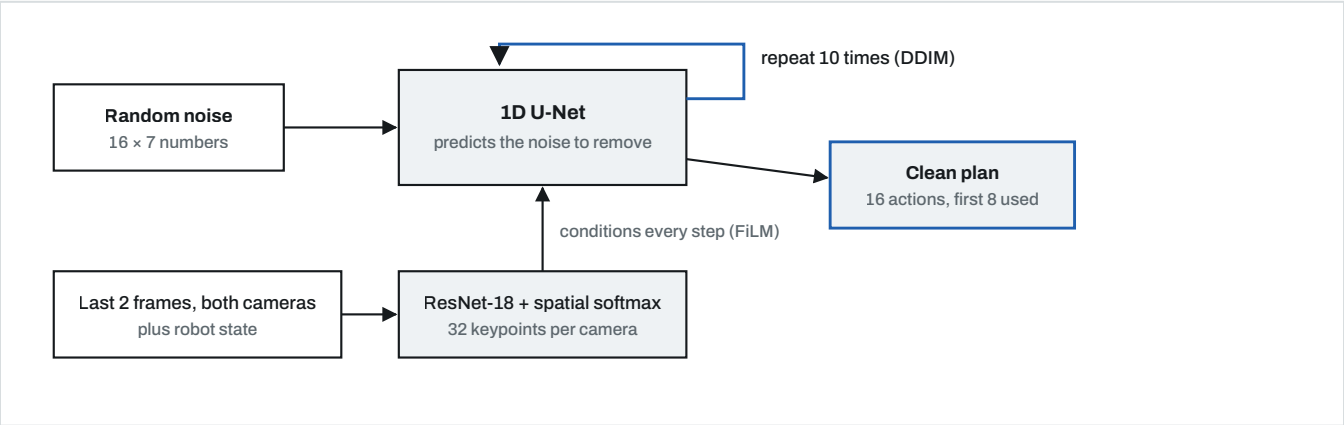


ACT as configured here. The dashed part exists only during training; at test time the style is fixed at zero, which makes the policy deterministic.

Diffusion Policy

Diffusion Policy (Chi et al., 2023) borrows the idea behind image generators. During training, random noise is added to a demonstrated chunk of 16 actions at one of 100 noise levels, and a network learns to predict that noise given the last two camera frames and the robot state. At test time the policy starts from pure random numbers and repeatedly subtracts the predicted noise. Here 10 denoising steps with the DDIM sampler turn noise into a clean 16-action plan, of which 8 are executed before planning again.

Why go to this trouble? A network that outputs one answer (like ACT with $z = 0$) must pick a single average action. Denoising can produce any one of several valid answers depending on the starting noise, so it represents multimodal behaviour without averaging. The price is **randomness at test time**: different starting noise gives a different plan, so the result of an episode depends on the random seed. Section 6 shows how large that effect was.



Diffusion Policy as configured here (89M parameters, 128 px images with random 90 % crops). The starting noise is the only random input, and it decides which of several possible plans comes out.

SmolVLA: a vision-language-action model

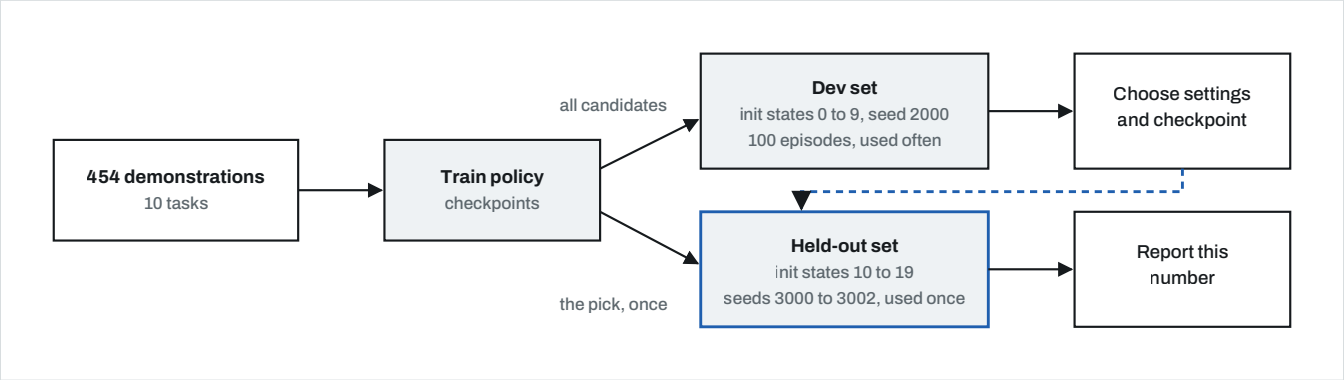
A **vision-language-action model** (VLA) starts from a model pretrained on images and text, so it already knows what "ketchup" and "basket" look like, and adds a part that outputs robot actions. SmolVLA (Shukor et al., 2025) uses the SmolVLM2 backbone and an **action expert** trained with **flow matching**, a close relative of diffusion: it learns a velocity field that carries random noise to an action chunk in 10 integration steps. It reads the task instruction as text, predicts 50 actions per chunk and, like Diffusion Policy, is random at test time. The checkpoint used here was published by its authors after training on the LeRobot LIBERO dataset, which covers all four LIBERO suites. It was evaluated, not retrained.

> **Why no language for ACT and Diffusion Policy?**
>
> In LIBERO-Object every task has a different set of objects on the table (checked in the task definition files). The scene alone identifies the task, so a vision-only policy trained on all ten tasks can work out what to pick. The videos confirmed it: ACT always went for the correct object, even when it then missed the grasp.

---

4. Evaluating fairly

Most of the effort in this project went into making the numbers trustworthy. Four rules did most of the work.



Every decision (which checkpoint, replanning interval, augmentation) was made on the dev set. The held-out scenes were scored once, with the chosen setting, so their number is not inflated by trying many options.

1. Separate the data you decide on from the data you report

If you try ten settings and report the best score on the same episodes, you have partly measured luck. Here the dev set (init states 0 to 9) chose everything, and the held-out set (init states 10 to 19) was scored once at the end. This mattered in practice: ACT without augmentation scored 89/100 on dev but 74/100 on held-out. The dev number was the best of several checkpoints and a favourable draw.

A related habit: decide the selection rule before seeing the results. For Diffusion Policy the rule "best dev score, ties go to the earliest checkpoint" was written into the log before the 75k and 100k checkpoints were scored.

2. Make every episode reproducible

An episode is fixed by its task, its init state and its **seed**, the number that drives every random choice in the simulator and the policy. Re-seeding all random generators before each episode means episode (task 3, init 14, seed 3000) is exactly the same situation for every policy. Seeding once at the start instead lets earlier episodes change the random state of later ones, so two policies never face quite the same episodes. One run in this project used the wrong setting by mistake; it reported 97/100 where the correct protocol gave 85/100 for the same model.

3. Random policies need several seeds

ACT is deterministic, so the seed does not matter for it: 99, 99 and 99 across three seeds, with the same episodes failing each time. Diffusion Policy and SmolVLA draw random noise when they act. With one seed, every episode reuses a single sequence of noise draws, and an unlucky sequence can sink many episodes of one task at once. Diffusion Policy scored **81, 96 and 99** on the same 100 scenes with seeds 3000, 3001 and 3002. A single-seed comparison could have ranked it anywhere.

4. Compare policies on the same episodes, with the right test

When two policies run on identical episodes, most episodes are solved by both or failed by both and say nothing about which is better. Only the **discordant** episodes matter: those where exactly one policy succeeds. **McNemar's exact test** asks whether the split of discordant episodes is more lopsided than a fair coin would give.

> Worked example: ACT vs Diffusion Policy, 300 held-out episodes
>
> ACT succeeded on 23 episodes that Diffusion Policy failed; Diffusion Policy succeeded on 2 that ACT failed. If the two were equally good, each of these 25 discordant episodes would be a coin flip. The chance of a split at least as lopsided as 23 to 2 is
>
> `p = 2 × P(X ≤ 2), X ~ Binomial(25, 0.5) = 2 × 326 / 33,554,432 ≈ 0.00002`
>
> So a real difference is all but certain. For ACT vs SmolVLA the split is 23 to 3 (p = 0.0001). Because three pairs were compared, a strict reader divides the usual 0.05 threshold by three (Bonferroni correction, 0.0167); both comparisons clear it easily.

How big is the noise? With 100 episodes and a true success rate of 85 %, the standard error is $\sqrt{(0.85 \times 0.15 / 100)} \approx 3.6$ percentage points. Two checkpoints of the same run scoring 89 and 82 can easily be the same policy. That is why single-checkpoint "improvements" of a few points were never trusted without a paired test.

5. Make the computation itself reproducible

Re-seeding fixes the random draws, but the GPU can still compute the same network slightly differently from run to run. PyTorch's `cudnn.benchmark` setting times several convolution algorithms on the first batch and keeps the fastest; which one wins can change between runs, and different algorithms round differently. In a single prediction that difference is invisible, but in a closed-loop rollout a gripper a fraction of a millimetre off changes the next image, and after a few hundred steps the episode can end differently. This project found it when re-running the final evaluation to record videos: ACT scored 99 where the saved run said 97, and the 10- and 20-demo models moved by 2 points each. With deterministic algorithms switched on, repeat runs matched episode for episode, and every reported number was re-measured that way. The ranking did not change. The practical habit: run the same command twice before trusting a difference of a couple of points.

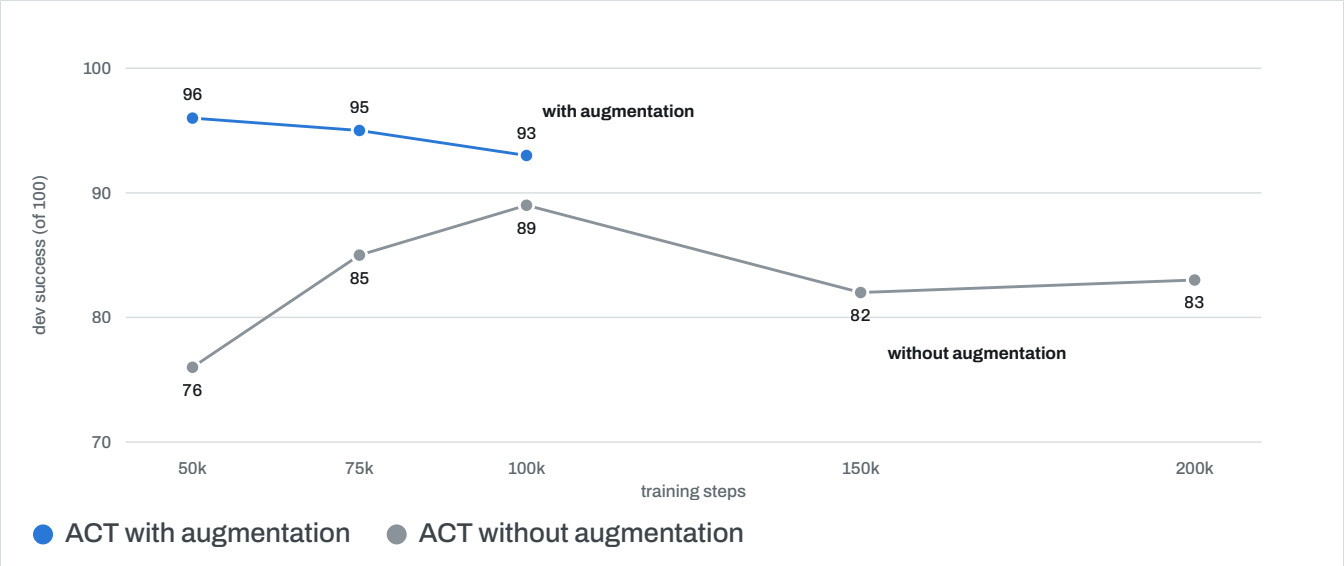
---

5. What happened, step by step

The project ran as a series of cycles: measure, look at the failures on video, change one thing, measure again on the same episodes. This table is the short version; the full record is in `results/log.md` in the repository.

Cycle	Question	What was done	Result
0	Can the evaluation be trusted?	Evaluated the published SmolVLA checkpoint, which reports 96 %	Only 56 % at first. A newer MuJoCo renders the floor about 28 % darker than the training images; pinning MuJoCo 3.3.2 gave 72 %
1	Why is SmolVLA still below 96 %?	Videos: the grasp misses and the arm carries on with the place motion. Replanned every 10 actions instead of executing all 50	78 → 94 on dev ($p = 0.0015$); 97/100 on held-out at the time, matching the paper
2	Can ACT learn one task?	ACT on alphabet soup only (44 demos), 50 init states	0/50 at first. Rendering at 256 px like the training frames: 5/50. Replanning: 38/50. Final checkpoint: **50/50**
3	One ACT for all ten tasks	454 demos, 100k steps; dev at 50k, 75k, 100k	76, 85, 89 on dev, but **74/100** held-out. Fails on tall objects (milk, ketchup)
3	Can inference settings fix it?	Replan every 5; temporal ensembling	71 and 45 on dev. Neither helps
4, 5	Does more training help?	Extended to 200k steps; averaged the weights of late checkpoints	82, 83 and 83. Loss keeps falling, success does not: overfitting
6	Does augmentation fix overfitting?	Same run with random colour, shift and rotation on training images	96 on dev at 50k; **99/100** held-out (25 fixed, none broken)
7	Is Diffusion Policy better?	Same data and augmentation, 89M-parameter configuration	100/100 on dev, but 81, 96, 99 on held-out depending on the seed
8	How much data is needed?	ACT recipe on 10 and 20 demos per task	74 and 91 held-out, against 99 with all demos
9		Re-ran every final evaluation to record failure videos	ACT moved by 2 points between identical runs: GPU kernel autotuning. Made evaluation

Cycle	Question	What was done	Result
	Are the numbers reproducible?		deterministic and re-measured everything; repeat runs now match exactly



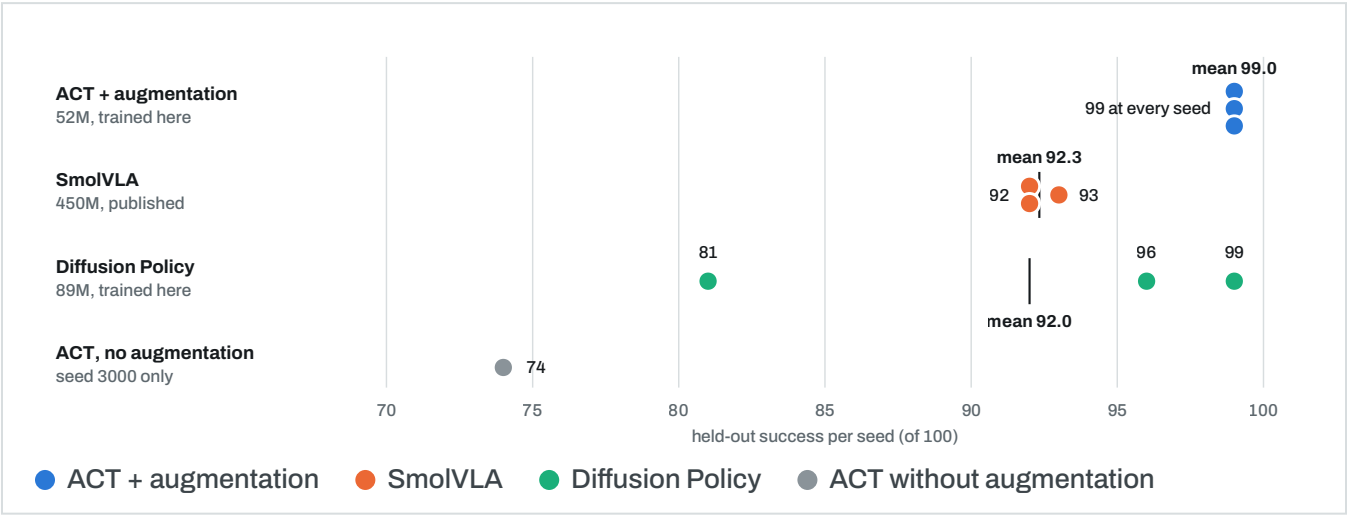
ACT, all ten tasks, dev set. Without augmentation, success peaks around 100k steps and then falls while the training loss keeps going down. With augmentation the policy is better at every checkpoint, already at 50k.



Same dev episode (milk, init state 3). Left: without augmentation the gripper hovers beside the carton until time runs out. Right: with augmentation it grasps the carton and places it.

6. Final results

Each policy was run on the same 100 held-out scenes three times, with seeds 3000, 3001 and 3002, every episode re-seeded: 300 identical episodes per policy.



Each dot is one seed (100 episodes); the vertical bar marks the mean of three seeds. ACT scores 99 at every seed. Diffusion Policy spans 18 points on identical scenes, which is the seed effect from section 4.

Policy	Seed 3000	Seed 3001	Seed 3002	Total
ACT + augmentation (52M, trained here)	**99**	**99**	**99**	**297 / 300 (99.0 %)**
SmolVLA (450M, published checkpoint)	93	92	92	277 / 300 (92.3 %)
Diffusion Policy + augmentation (89M, trained here)	81	96	99	276 / 300 (92.0 %)
ACT without augmentation	74			74 / 100

Paired comparison (300 shared episodes)	First wins	Second wins	p	Reading
ACT + aug vs Diffusion Policy	23	2	0.00002	Clear difference
ACT + aug vs SmolVLA	23	3	0.0001	Clear difference, also after correcting for 3 tests
Diffusion Policy vs SmolVLA	21	22	1.0	Cannot be told apart
ACT + aug vs ACT without (seed 3000)	25	0	< 0.0001	Augmentation clearly helps

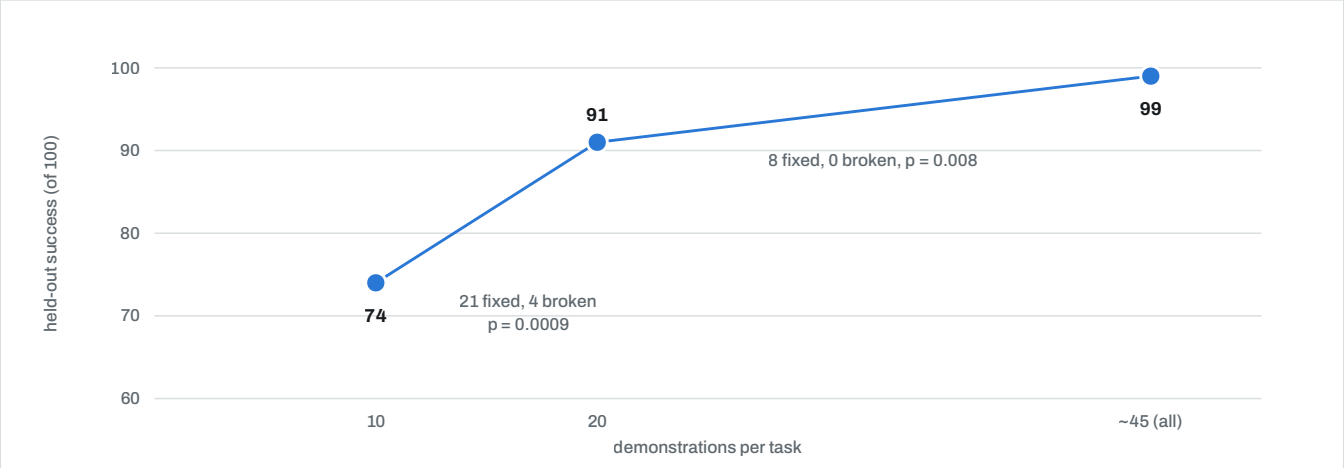
How to read this result

A 52M-parameter model trained on one laptop GPU beat a 450M-parameter VLA on this suite. That does not mean small models are better in general. ACT here is a specialist trained only on these ten tasks, deterministic, and run with the replanning and augmentation that the experiments showed it needs. SmolVLA is a generalist trained on all four LIBERO suites and was not tuned for this one. On a harder suite, with longer tasks or instructions that only language can disambiguate, the ranking could change. The defensible claim is narrower and still strong: on LIBERO-Object, a carefully trained small policy is more reliable than the published VLA.

---

7. How much data is enough

The winning ACT recipe was retrained on random subsets of 10 and 20 demonstrations per task. The subsets are nested (the 10 are part of the 20), so the only change is how much data the policy saw. The checkpoint was chosen on dev and scored once on held-out.



ACT with augmentation, held-out seed 3000. Going from 10 to 20 demonstrations per task is a large, clear gain; going from 20 to all of them adds 8 more points, fixing 8 episodes and breaking none (p = 0.008).

Demos per task	Dev at 25k / 50k steps	Held-out	Weakest tasks
10 (100 total)	75 / 68	74	Cream cheese 4/10, pudding 5/10, orange juice 6/10
20 (200 total)	86 / 91	91	Cream cheese, bbq sauce, ketchup 8/10 each
~45 (454 total)	96 at 50k	99	Ketchup 9/10

Three things to take from this study.

- **Diminishing returns.** Most of the gain arrives by 20 demonstrations per task (+17 points). The last 25 demonstrations per task add 8 more: they buy reliability, not basic competence.
- **Less data overfits sooner.** With 10 demos the 50k checkpoint is worse than the 25k one (68 vs 75 on dev). With 20 demos the 50k checkpoint is still better, so that run may even be slightly under-trained. The right training length depends on the dataset size.
- **The failure type changes.** With few demos the weak tasks are small boxes (cream cheese, pudding), often partly hidden behind taller objects. With all demos the one remaining failure is a tall bottle slipping out of the gripper. Collecting more demos of exactly the hard situations would be the efficient next step.

One limitation applies: each data size was trained once, so part of each difference may be training noise.

---

8. How the policies fail

Every failure discussed here was checked by watching the rollout, not inferred from numbers. Four patterns cover almost all of them.



Slip after lifting. The final ACT model's only failure: ketchup, held-out init 11, identical at every seed. The bottle is lifted (frame 3), slips out during the carry, and the arm carries on to the basket with nothing in it (frame 4).



Knocked over. Diffusion Policy, bbq sauce, held-out init 17 (seed 3000). The approach tips the tall bottle over; the policy never saw a fallen bottle in training and hovers until time runs out.



Target hidden behind another object. ACT trained on only 10 demos per task, pudding, held-out init 12. The small box sits behind the ketchup bottle; the arm hovers over the bottle and never reaches past it.



Released short. ACT without augmentation, ketchup, held-out init 10. The grasp and carry work, but the bottle is let go beside the basket. With augmentation the same episode succeeds.

> The common thread: no recovery
>
> None of the policies ever tries again after a missed grasp. They imitate demonstrations, and the demonstrations never contain a failed grasp followed by a retry, so "try again" is a behaviour none of them has seen. Fixing this needs data, not architecture: demonstrations that include recoveries, or methods such as DAgger that collect corrections in the states the policy actually reaches.

---

9. Lessons that transfer

1. **Reproduce a published number before training anything.** The first evaluation of a public checkpoint gave 56 % against a published 96 %. Finding out why (simulator version, open-loop chunks) fixed problems that would otherwise have hit every model trained afterwards.
2. **Change one thing at a time, on the same episodes.** Every claimed improvement here is a paired comparison with everything else fixed.
3. **Watch the rollouts.** "Success went down" says nothing about why. The videos showed missed grasps, hovering, tipped bottles and hidden targets, and each pointed to a specific fix.
4. **Make evaluation inputs match training inputs.** Image size and renderer version each cost tens of points with no error message.

5. **When loss falls but success does not, you are overfitting.** Augmentation, not more training, was the fix (74 to 99).
6. **Replan, do not run long chunks blind.** Executing 10 of the predicted actions instead of the whole chunk lifted SmolVLA from 78 to 94 and single-task ACT from 5 to 38 of 50.
7. **Treat random policies as random.** Evaluate them with several seeds and re-seed every episode, or the ranking is partly luck.
8. **Fix the selection rule before looking.** A dev set for choices, a held-out set used once, and tie rules written down in advance.
9. **Run it twice.** GPU kernel autotuning made identical runs differ by 2 points in 100. Deterministic kernels, and a repeat run that matches, come before any claim about a small difference.
10. **Write down mistakes.** This project had a seeding slip, a monitoring script that never fired and a lost disk. Each is recorded in the log with what it changed, which is what makes the final numbers believable.

---

10. Interview questions

Questions an interviewer is likely to ask about this project, with short answers to practise from. Say them in your own words.

Why did a 52M ACT model beat a 450M VLA?

On this suite ACT is a specialist: trained only on these ten tasks, deterministic at test time, and run with the two fixes the experiments found (replanning every 10 actions and image augmentation). SmolVLA is a generalist trained on four suites and is random at test time. Over 300 paired episodes ACT reached 99.0 % against 92.3 % ($p = 0.0001$). I would not generalise this beyond LIBERO-Object without testing harder suites.

What is action chunking, and how should a chunk be executed?

Predicting many future actions at once. It shortens the effective horizon and handles pauses in human demos. Executing the whole chunk blind misses failed grasps, and temporal ensembling blurred the late corrections (45/100). Replanning every 10 actions was best for every model.

The published SmolVLA scored 56 % in your first test. How did you find the cause?

One variable at a time. Environment reuse and seeds made no difference. Then I bisected MuJoCo versions with a render probe: from 3.3.3 the floor renders about 28 % darker than in the training images. Pinning 3.3.2 gave 72 %. Videos then showed missed grasps during open-loop chunks, and replanning brought it to the published level on held-out scenes.

Training loss kept going down but success did not. What did you do?

Recognised it as overfitting to pixel positions, checked that more training and weight averaging did not help (82 to 83), and added image augmentation. Held-out success went from 74 to 99, with 25 episodes fixed and none broken.

How did you compare two policies statistically?

Both run on identical episodes (same task, init state and per-episode seed). Only discordant episodes matter, and McNemar's exact test checks whether their split is more lopsided than a coin flip. With three comparisons I also note the Bonferroni threshold.

Why did Diffusion Policy need several seeds?

It samples starting noise, so outcomes depend on the seed. With per-episode re-seeding a single seed repeats one noise sequence everywhere; it scored 81, 96 and 99 on the same scenes. ACT is deterministic and scored 99 at every seed.

How did you avoid choosing on the test set?

A dev set (init states 0 to 9) for every decision, a held-out set (10 to 19) scored once, and the checkpoint rule written down before scoring. Without augmentation the gap between dev 89 and held-out 74 showed why this matters.

How much data does the policy need?

10 demos per task gave 74 %, 20 gave 91 %, all ~45 gave 99 %. Most of the value comes in the first 20; the rest buys reliability on the hard cases.

ACT's CVAE vs diffusion: how do they handle inconsistent demonstrations?

ACT learns a style variable during training so it does not average styles, then fixes it to zero at test time and outputs one deterministic plan. Diffusion learns the whole distribution of plans and samples one, which handles several valid answers but makes the policy random.

Something that went wrong, and how you handled it?

One Diffusion Policy evaluation used the wrong seeding flag and reported 97/100. When I re-ran it correctly it gave 85, so I investigated, found the seed effect, evaluated all three policies with three seeds and corrected the published README and website. Later, re-running to record videos, ACT moved by 2 points between identical runs; I traced it to GPU kernel autotuning, made evaluation deterministic and re-measured every number. Both are recorded in the log.

What would you do next?

Train each policy with several training seeds, add a harder suite such as LIBERO-10, and attack the no-retry problem with recovery demonstrations or DAgger-style corrections. On a real robot: match camera placement and resolution, measure latency, and keep replanning.

How would this transfer to a real robot?

The same lessons apply with more force: inputs must match training (camera pose, resolution, lighting, so augmentation matters more), replanning compensates for slower and noisier control, and evaluation needs many trials per setting because every trial costs time.

11. Glossary

Policy	The network that maps what the robot observes to the action it takes.
Behaviour cloning	Training a policy by supervised learning on demonstrated observation and action pairs.
Episode, rollout	One attempt at a task, from the starting layout until success or the time limit.
Init state	One of LIBERO's fixed starting layouts of the objects for a task.
Seed	The number that fixes every random choice; re-seeding before each episode makes it reproducible.
Compounding error	Small mistakes leading to unseen states where the policy makes bigger mistakes.
Multimodality	Several different correct actions for the same situation, such as grasping from the left or the right.
Action chunk	A block of future actions predicted at once.
Open loop, closed loop	Executing without looking at new observations, versus correcting with each new observation.
Replanning	Executing only the start of a chunk, then predicting a new chunk.
Temporal ensembling	Averaging all the overlapping chunk predictions for the current step.
CVAE	Conditional variational autoencoder; in ACT it learns a style variable from training demonstrations.
KL penalty	A loss term keeping a learned latent close to a standard normal distribution.
Diffusion, denoising	Generating data by starting from noise and repeatedly removing predicted noise.

DDPM, DDIM	Two samplers for diffusion models; DDIM needs far fewer steps (10 here).
Flow matching	Learning a velocity field that moves noise to data; used by SmolVLA's action expert.
VLM, VLA	Vision-language model; vision-language-action model, a VLM extended to output robot actions.
FiLM	Feature-wise modulation: conditioning one network on another by scaling and shifting its features.
Spatial softmax	Turns an image feature map into the coordinates of a few keypoints.
Image augmentation	Random changes to training images so the policy cannot memorise exact pixels.
Overfitting	Fitting the training data better while doing worse on new situations.
Dev and held-out sets	Episodes for making decisions, and episodes used once to report the final number.
McNemar's test	A test for two classifiers or policies on the same items, using only the items where they disagree.
Bonferroni correction	Dividing the significance threshold by the number of comparisons made.
AMP	Automatic mixed precision: training partly in 16-bit floats, about 45 % faster here.

---

12. Further reading

- Zhao, Kumar, Levine, Finn (2023). Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware. The ACT paper.
- Chi et al. (2023). Diffusion Policy: Visuomotor Policy Learning via Action Diffusion.
- Shukor et al. (2025). SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics.
- Liu et al. (2023). LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning.
- Ho, Jain, Abbeel (2020). Denoising Diffusion Probabilistic Models. Song, Meng, Ermon (2020). Denoising Diffusion Implicit Models.
- Lipman et al. (2022). Flow Matching for Generative Modeling.
- Ross, Gordon, Bagnell (2011). A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. The DAgger paper, on compounding errors.

- Agarwal et al. (2021). Deep Reinforcement Learning at the Edge of the Statistical Precipice. On reporting results with few runs and seeds.
- Hugging Face LeRobot, the library used for all training and evaluation here.

All numbers come from the evaluation files in the repository (`results/eval/`) and the experiment log (`results/log.md`). Simulation only: LIBERO-Object, MuJoCo 3.3.2, LeRobot 0.6.1, one RTX 3060 laptop GPU (6 GB).